# P0476r0
# Bit-casting object representations

Published Proposal, 16 October 2016

**This version:**
http://wg21.link/P0476r0

**Author:**
JF Bastien (Apple)

**Audience:**
LEWG, LWG

**Project:**
ISO JTC1/SC22/WG21: Programming Language C++

**Source:**
github.com/jfbastien/papers/blob/master/source/P0476r0.bs

**Implementation:**
github.com/jfbastien/bit_cast/

---

## Abstract

Obtaining equivalent object representations The Right Way™.

## Table of Contents



## § 1. Background

Low-level code often seeks to interpret objects of one type as another: keep the same bits, but obtain an object of a different type. Doing so correctly is error-prone: using `reinterpret_cast` or `union` runs afoul of type-aliasing rules yet these are the intuitive solutions developers mistakenly turn to.

Attuned developers use `aligned_storage` with `memcpy`, avoiding alignment pitfalls and allowing them to bit-cast non-default-constructible types.

This facility inevitably ends up being used incorrectly on pointer types, we propose using appropriate concepts to prevent misuse. As our sample implementation demonstrates we could as well use `static_assert` or template SFINAE, but the timing of this library feature will likely coincide with concept's standardization.

Furthermore, it is currently impossible to implement a `constexpr` bit-cast function, as `memcpy` itself isn't `constexpr`. Marking our proposed function as `constexpr` doesn't require or prevent `memcpy` from becoming `constexpr`. This leaves implementations free to use their own internal solution (e.g. LLVM has a `bitcast` opcode).

We propose to standardize this oft-used idiom, and avoid the pitfalls once and for all.

## 2. Proposed Wording

Below, substitute the `�` character with a number the editor finds appropriate for the sub-section.

### 2.1. Synopsis

Under 20.2 Header `<utility>` synopsis [**utility**]:

```
namespace std {
  // ...

  // 20.2.� bit-casting:
  template<typename To, typename From>
  requires
    sizeof(To) == sizeof(From) &&
    is_trivially_copyable_v<To> &&
    is_trivially_copyable_v<From> &&
    is_standard_layout_v<To> &&
    is_standard_layout_v<From> &&
    !(is_pointer_v<From> &&
      is_pointer_v<To>) &&
    !(is_member_pointer_v<From> &&
      is_member_pointer_v<To>) &&
    !(is_member_object_pointer_v<From> &&
      is_member_object_pointer_v<To>) &&
    !(is_member_function_pointer_v<From> &&
      is_member_function_pointer_v<To>)
  constexpr To bit_cast(const From& from) noexcept;

  // ...
}
```

### 2.2. Details

Under 20.2.`�` Bit-casting [**utility.bitcast**]:

```
template<typename To, typename From>
requires
  sizeof(To) == sizeof(From) &&
  is_trivially_copyable_v<To> &&
  is_trivially_copyable_v<From> &&
  is_standard_layout_v<To> &&
  is_standard_layout_v<From> &&
  !(is_pointer_v<From> &&
    is_pointer_v<To>) &&
  !(is_member_pointer_v<From> &&
    is_member_pointer_v<To>) &&
  !(is_member_object_pointer_v<From> &&
    is_member_object_pointer_v<To>) &&
  !(is_member_function_pointer_v<From> &&
    is_member_function_pointer_v<To>)
constexpr To bit_cast(const From& from) noexcept;
```

1. Requires: `sizeof(To) == sizeof(From)`, `is_trivially_copyable_v<To>` is `true`, `is_trivially_copyable_v<From>` is `true`, `is_standard_layout_v<To>` is `true`, `is_standard_layout_v<From>` is `true`, `is_pointer_v<To> && is_pointer_v<From>` is `false`, `is_member_pointer_v<To> && is_member_pointer_v<From>` is `false`, `is_member_object_pointer_v<To>`

`&& is_member_object_pointer_v<From>` is `false`, `is_member_function_pointer_v<To> && is_member_function_pointer_v<From>` is `false`.

2. Returns: an object of type `To` whose *object representation* is equal to the object representation of `From`. If multiple *object representations* could represent the *value representation* of `From`, then it is unspecified which `To` value is returned. If no *value representation* corresponds to `To`'s *object representation* then the returned value is unspecified.

### § 2.3. Feature testing

The `__cpp_lib_bit_cast` feature test macro should be added.

## § 3. Appendix

The Standard's **[basic.types]** section explicitly blesses `memcpy`:

> For any trivially copyable type `T`, if two pointers to `T` point to distinct `T` objects `obj1` and `obj2`, where neither `obj1` nor `obj2` is a base-class subobject, if the *underlying bytes* (1.7) making up `obj1` are copied into `obj2`, `obj2` shall subsequently hold the same value as `obj1`.
>
> [*Example:*
>
> ```
> T* t1p;
> T* t2p;
> // provided that t2p points to an initialized object ...
> std::memcpy(t1p, t2p, sizeof(T));
> // at this point, every subobject of trivially copyable type in *t1p contains
> // the same value as the corresponding subobject in *t2p
> ```
>
> — *end example*]

Whereas section [class.union] says:

> In a union, at most one of the non-static data members can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time.

## § 4. Acknowledgement

Thanks to Saam Barati, Jeffrey Yasskin, and Sam Benzaquen for their early review and suggested improvements.